



Garrido

En esta tarea vas a desarrollar una aplicación en **Java** que simule el **uso y la recarga de un vehículo eléctrico**, aplicando los principios básicos de la **programación orientada a objetos**.

El ejercicio tiene como objetivo reforzar el uso de **clases, atributos, métodos, operadores y objetos**, evitando el uso de estructuras condicionales clásicas (if, else, switch) y empleando en su lugar **expresiones auxiliares**, el **operador ternario** y los métodos Math.min() y Math.max().

Crear las clases dentro de un paquete de nombre ejercicio1.

Clase VehiculoElectrico

Debes crear una clase llamada VehiculoElectrico que represente un vehículo eléctrico.

Atributos

La clase deberá incluir, al menos, los siguientes atributos:

- modelo: modelo del vehículo.
- bateria: nivel actual de batería en porcentaje (0 a 100).
- consumoKm: porcentaje de batería que se consume por cada kilómetro recorrido.
- kilometrosRecorridos: número total de kilómetros recorridos por el vehículo.

Constructor

La clase dispondrá de un **constructor con parámetros** que reciba:

- El modelo del vehículo.
- El consumo por kilómetro.

Al crear el objeto:

- La batería debe inicializarse al **100%**.
- Los kilómetros recorridos deben inicializarse a **0**.

Métodos

La clase VehiculoElectrico deberá implementar los siguientes métodos:

recorrer(double kilometros)

Simula un trayecto del vehículo durante un número determinado de kilómetros.

- El consumo se calcula como:
kilometros * consumoKm
- La batería se reduce según el consumo.
- El nivel de batería nunca puede ser negativo.
- Los kilómetros recorridos se acumulan.

El control del valor mínimo de la batería debe realizarse utilizando Math.max().

recargar(double porcentaje)

Aumenta el nivel de batería en el porcentaje indicado.

- El nivel de batería no puede superar el 100%.
- Debe utilizarse Math.min() para controlar el límite máximo.

estadoBateria()

Devuelve una cadena de texto indicando el estado de la batería del vehículo:

- "BUENA" si la batería es mayor o igual al 60%.



- "MEDIA" si la batería está entre el 20% y el 59%.
- "CRÍTICA" si la batería es inferior al 20%.

Este método **debe implementarse exclusivamente utilizando el operador ternario**, sin utilizar la estructura if.

mostrarInformacion()

Muestra por consola toda la información actual del vehículo:

- Modelo
- Nivel actual de batería
- Kilómetros totales recorridos
- Estado de la batería

Clase Principal

Debes crear una clase Principal que contenga el método main, encargado de coordinar la ejecución del programa.

El método main deberá:

1. Crear un objeto VehiculoElectrico con valores definidos directamente en el código.
2. Solicitar al usuario, mediante la clase Scanner:
 - El número de kilómetros que se recorrerán.
 - El porcentaje de batería que se desea recargar.
3. Simular el trayecto llamando al método recorrer(...).
4. Aplicar la recarga llamando al método recargar(...).
5. Mostrar la información final del vehículo utilizando el método mostrarInformacion().

Criterios de calificación

Clase VehiculoElectrico	Puntos
Creación correcta de atributos	0.5
Creación correcta del constructor	0.5
Creación correcta del método recorrer	0.5
Creación correcta del método recargar	0.5
Creación correcta del método estadoBateria	0.5
Total	2.5

Criterio	Puntos
Creación correcta del objeto	0.5
Se leen correctamente los datos solicitados	0.5
Simulación del trayecto.	0.5
Aplica correctamente la recarga del vehículo	0.5
Muestra correctamente la información utilizando el método mostrarInformacion	0.5
Total	2.5

El programa compila y el resultado funciona bien en general (constructor correcto, recargar con Math.min, estadoBateria con ternario y main lee datos, simula trayecto, recarga y muestra información). El control de batería en recorrer no sigue la fórmula indicada con Math.max para limitar el mismo.

Nota 4,75

Clase VehiculoElectrico (2,5 puntos)

Creación correcta de atributos:

0,5 Atributos correctos y con modificadores private.

Creación correcta del constructor:

0,5 Inicializa batería al 100% y kilómetros a 0, y asigna modelo y consumoKm. Correcto.

Creación correcta del método recorrer:

0,25 Se calcula el consumo como kilometros * consumoKm y se acumulan kilómetros, pero se incumple el



requisito de evitar if/else y no se usa Math.max como pide el enunciado para garantizar que la batería no sea negativa. Además, la condición `if (bateria == Math.max(bateria, consumo))` no es una comprobación correcta de "batería suficiente" (lo correcto sería comparar `bateria >= consumo`).

Creación correcta del método recargar:

0,5 Aplica Math.min para limitar a 100 y suma el porcentaje correctamente. El if de porcentaje negativo no se pedía y rompe la restricción de "sin if", pero funcionalmente el método de recarga está bien.

Creación correcta del método estadoBateria:

0,5 Correcto: devuelve "BUENA/MEDIA/CRÍTICA" con operador ternario y rangos adecuados.

Clase Principal (2,5 puntos)

Creación correcta del objeto:

0,5 Se crea el objeto con valores definidos en el código.

Se leen correctamente los datos solicitados:

0,5 Se leen kilómetros y recarga con Scanner. No se pide, validar el formato numérico con try/catch.

Simulación del trayecto:

0,5 Se llama a recorrer(kms), correcto.

Aplica correctamente la recarga del vehículo:

0,5 Se llama a recargar(recarga), correcto.

Muestra correctamente la información utilizando el método mostrarInformacion:

0,5 Se llama a mostrarInformacion y muestra los datos finales.



RA3. Ejercicio práctico 2. Puntuación máxima 10 puntos.

Escribe un programa en **Java**, en una **única clase** llamada **Ejercicio2.java**, que solicite al usuario un **número entero positivo n** mediante entrada estándar.

A continuación, el programa deberá mostrar un **menú de opciones** con las siguientes funcionalidades:

1. **Sumar los números pares** comprendidos entre 1 y n.
2. **Sumar los múltiplos de un número k** (introducido por el usuario) comprendidos entre 1 y n.
3. **Mostrar los números no primos** comprendidos entre 1 y n.
4. **Salir del programa.**

El usuario elegirá una opción del menú y el programa realizará el cálculo correspondiente, mostrando el resultado por pantalla.

El **menú deberá repetirse** continuamente hasta que el usuario seleccione la opción **Salir**.

Estructura del programa

- El programa debe estar implementado en **una única clase** llamada Ejercicio2.
- Cada opción del menú debe desarrollarse mediante un **método independiente**, encargado de realizar el cálculo correspondiente.
- El método main será el encargado de:
 - Solicitar el valor de n.
 - Mostrar el menú.
 - Leer la opción elegida por el usuario.
 - Llamar al método correspondiente según la opción seleccionada.
- El programa debe validar que los valores introducidos por el usuario sean correctos, valores numéricos enteros y controlar los rangos (por ejemplo, que n y k sean números enteros mayores que 0).

Ayuda para la implementación

- **Números primos:** Son números naturales mayores que 1 que solo tienen dos divisores exactos: el 1 y ellos mismos. Ejemplos: 2, 3, 5, 7, 11... **Los números no primos** son los que tienen un divisor distinto de 1 y ellos mismos: 4, 6, 8, 9...

Salida del programa

- En las opciones de suma, el programa mostrará el **resultado total** del cálculo.
- En la opción de mostrar número primos, el programa mostrará todos los números encontrados hasta n, separados por espacios.

Criterios de calificación

Criterio	Puntos
Correcta creación y gestión del menú.	2
Suma correctamente los números pares entre 1 y n y muestra el resultado.	2
Suma los múltiplos de un número k entre 1 y n y muestra el resultado.	2
Muestra los números no primos hasta n.	2
Cada opción del menú (1, 2 y 3) se desarrolla mediante un método independiente .	1
Se validan correctamente los datos de entrada.	1
Total	10

El programa compila y el menú se gestiona bien (se repite hasta salir, métodos separados y validación de



enteros). La suma de pares es correcta. Sin embargo, la opción 2 está mal interpretada (suma $k \cdot i$ en lugar de sumar los múltiplos de k dentro de $1..n$) y la opción 3 falla porque el método `esPrimo` considera 0 y 1 como primos, por lo que la lista de “no primos” será incorrecta (no incluye 1 y puede incluir mal otros).

Nota 7,75

Correcta creación y gestión del menú:

2,0 Menú correcto: se repite hasta elegir 4, y cada opción llama a su método.

Suma correctamente los números pares entre 1 y n y muestra el resultado:

2,0 Correcto recorre de 0 a n de 2 en 2 y suma (incluir 0 no afecta).

Suma los múltiplos de un número k entre 1 y n y muestra el resultado :

0 La implementación no cumple el enunciado: se calcula la suma de la serie $k \cdot 0 + k \cdot 1 + \dots + k \cdot n$ (que llega hasta $k \cdot n$), pero eso no es “sumar los múltiplos de k comprendidos entre 1 y n ”. Lo correcto sería sumar los i del 1 al n tales que $i \% k == 0$.

Muestra los números no primos hasta n :

1,75 La idea de recorrer y usar `esPrimo` está bien. Se imprime uno por línea; el enunciado pide separados por espacios.

Cada opción del menú (1, 2 y 3) se desarrolla mediante un método independiente:

1,0 Sí: `sumarLosPares`, `sumarMultiplos`, `mostrarNoPrimos` (y `esPrimo` auxiliar).

Se validan correctamente los datos de entrada:

1,0 Se valida que n y k sean enteros (`try/catch`) y se fuerza $n > 0$ y $k > 0$. Correcto.



RA4. Ejercicio práctico 3. Puntuación máxima 10 puntos.

Desarrollar una aplicación sencilla en Java que permita gestionar **productos de un inventario**

Estructura del proyecto

El programa debe organizarse en **dos paquetes**:

- modelo, contendrá la clase Producto
- principal, contendrá la clase Principal con el método main

Clase Producto (paquete modelo)

Atributos

La clase Producto debe incluir los siguientes atributos:

- nombre
- categoria
- precio
- disponible, indica si el producto está disponible o no.

Además, debe existir un atributo **estático**:

- private static int totalProductos, es un contador del número total de los productos creados.

Constructores

La clase deberá implementar **dos constructores**:

1. Constructor completo

Producto(String nombre, String categoria, double precio)

- Inicializa todos los atributos.
- El producto se crea como **disponible**.
- Incrementa el contador totalProductos.

2. Constructor simplificado

Producto(String nombre, String categoria)

- Inicializa el nombre y la categoría.
- Asigna un precio por defecto de 0.0.
- El producto se crea como **no disponible**.
- Incrementa el contador totalProductos.

Métodos de instancia

La clase Producto deberá incluir los siguientes métodos:

- public void activarProducto(), cambia el estado del producto a disponible.
- public void desactivarProducto(), cambia el estado del producto a no disponible.
- public void cambiarPrecio(double nuevoPrecio), modifica el precio del producto.
- public String obtenerDisponibilidad(), devuelve "DISPONIBLE" o "NO DISPONIBLE" según el estado del producto.
- public String toString(), devuelve una cadena con todos los datos del producto, excepto información interna adicional.



Métodos estáticos

La clase debe incluir el siguiente método estático:

- `public static int getTotalProductos()`, devuelve el número total de productos creados.

Clase Principal (paquete principal)

La clase Principal debe contener el método `main` y realizar las siguientes acciones:

1. Crear un producto usando el **constructor completo**.
2. Crear un segundo producto usando el **constructor simplificado**.
3. Activar el segundo producto.
4. Cambiar el precio de uno de los productos.
5. Mostrar la información de ambos productos utilizando `toString()`.
6. Mostrar por pantalla el número total de productos creados usando el método estático.

Ejemplo de salida orientativa

Producto: Portátil

Categoría: Informática

Precio: 899.99 €

Estado: DISPONIBLE

Producto: Ratón

Categoría: Accesorios

Precio: 25.0 €

Estado: DISPONIBLE

Productos totales creados: 2

Criterios de calificación

Criterio	Puntos
Correcta creación de los atributos, modificadores de acceso y tipos.	0.5
Correcta creación del constructor completo	0.5
Correcta creación del constructor simplificado	0.5
Correcta creación de los métodos <code>activarProducto</code> y <code>desactivarProducto</code>	1
Correcta creación del método <code>cambiarPrecio</code>	1
Correcta creación del método <code>obtenerDisponibilidad</code>	1
La clase está incluida en el paquete indicado	0.5

Criterio	Puntos
Creación correcta de los productos	1
Activar correctamente el segundo producto	1
Cambia correctamente el precio	1
Muestra la información de ambos productos usando el método <code>toString</code>	1
Muestra el número total de productos creados	1
Total	10

El programa compila y realiza el flujo pedido en `main` (crea dos productos, activa el segundo, cambia precio y muestra información y total).

Nota 10

Clase Producto (5 puntos)

Correcta creación de los atributos, modificadores de acceso y tipos. 0,5 Correcto: atributos `private` y tipos correctos, contador `private static`.

Correcta creación del constructor completo. 0,5 Correcto: inicializa todo, `disponible=true` e incrementa el contador.

Correcta creación del constructor simplificado. 0,5 Correcto: `precio=0.0`, `disponible=false` e incrementa el contador.

Correcta creación de los métodos `activarProducto` y `desactivarProducto`. 1,0 Correctos.

Correcta creación del método `cambiarPrecio`. 1,0 Correcto.

Correcta creación del método `obtenerDisponibilidad`. 1,0 La funcionalidad es correcta (devuelve "DISPONIBLE/NO DISPONIBLE").

La clase está incluida en el paquete indicado. 0,5 Correcto.

Clase Principal (5 puntos)

Creación correcta de los productos. 1,0 Correcto: crea uno con constructor completo y otro con simplificado.

Activar correctamente el segundo producto. 1,0 Correcto.

Cambia correctamente el precio. 1,0 Correcto (cambia el precio del producto1).

Muestra la información de ambos productos usando el método `toString`. 1,0 Correcto: se imprimen ambos productos (antes y después).

Muestra el número total de productos creados. 1,0 Correcto: usa el método estático.